

Incident Management System [IMS]

Docker and Kubernetes Implementation Screenshots

This project implements a scalable Incident Management System using microservices architecture. The system processes high-volume signals, generates incidents, and provides real-time monitoring through a dashboard. It is deployed using Docker and Kubernetes (KIND).

Docker Implementation:

1) Running Containers

```
prakash@SRIDHARANS-LAPTOP18:~/IMS-Prakash$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS NAMES	PORTS
6629f12cccc00->8501/tcp	ims-prakash-dashboard	"streamlit run /app/..."	52 seconds ago	Up 7 seconds ims_dashboard	0.0.0.0:86
4efe33d554d700->8000/tcp	ims-prakash-backend	"uvicorn app.main:ap..."	52 seconds ago	Up 24 seconds (healthy) ims_backend	0.0.0.0:90
270cb8116726433->5432/tcp	timescale/timescaledb:2.16.1-pg16	"docker-entrypoint.s..."	53 seconds ago	Up 51 seconds (healthy) ims_postgres	0.0.0.0:55
1fd1f6ec004380->6379/tcp	redis:7-alpine	"docker-entrypoint.s..."	53 seconds ago	Up 51 seconds (healthy) ims_redis	0.0.0.0:16
8e5a20dc08d4018->27017/tcp	mongo:7	"docker-entrypoint.s..."	53 seconds ago	Up 51 seconds (healthy) ims_mongo	0.0.0.0:37
bd88f56b4f4f44881->6443/tcp,	kindest/node:v1.35.1	"/usr/local/bin/entr..."	2 hours ago	Up 2 hours ims-control-plane	127.0.0.1:
d6eed87c3fe0.0.0.0:8000->30080/tcp,	ims-prakash-locust	"locust -f /app/scri..."	4 hours ago	Up 4 hours ims_locust	8000/tcp,
0.0.0.0:8090->8089/tcp					

2) Backend Health Check

All services (PostgreSQL, MongoDB, Redis) are connected successfully.

Pretty-print 

```
{
  "status": "ok",
  "checks": {
    "postgres": true,
    "mongo": true,
    "redis": true
  }
}
```

3) Load Testing (Locust) Load testing shows system handling ~200+ requests/sec.

← → ↻ 🏠 ⓘ http://localhost:8089 🔍 ☆ 🗨️ 📄 🌐 ⚙️ Relaunch to update ⋮



HOST
http://backend:8000

STATUS
READY

RPS
0

FAILURES
0%

⚙️

Start new load test

Number of users (peak concurrency)

Ramp up (users started/second)

Host

Advanced options
▼

START

← → ↻ 🏠 ⓘ http://localhost:8089 🔍 ☆ 🗨️ 📄 🌐 ⚙️ Relaunch to update ⋮



HOST
http://backend:8000

STATUS
RUNNING

USERS
100

RPS
301

FAILURES
5%

EDIT

STOP

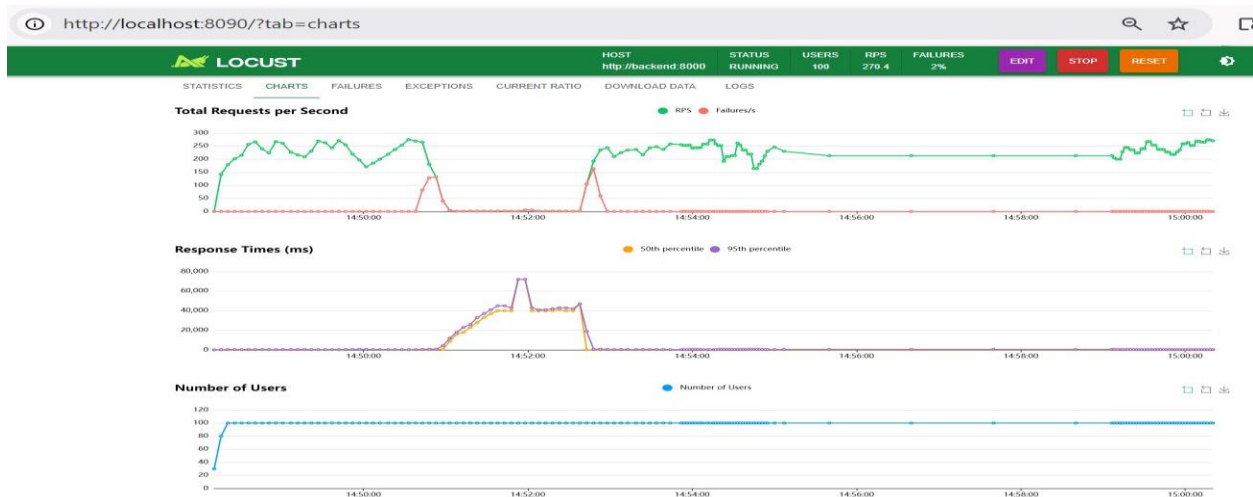
RESET

⚙️

STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA ! LOGS

⏸

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	GET /health	230	230	250	520	620	245.04	23	665	136	13.7	13.7
POST	POST /signals	4741	0	120	300	350	137.64	6	506	59.51	287.3	0
	Aggregated	4971	230	130	310	400	142.61	6	665	63.05	301	13.7



4) Kubernetes Implementation

The below Screenshots shows that all pods and services are running successfully in Kubernetes.

```
prakash@SRIDHARANS-LAPTOP18:~/IMS-Prakash$ kubectl get pods -n ims
```

NAME	READY	STATUS	RESTARTS	AGE
backend-cdbbdd7f-xtswp	1/1	Running	1 (4m15s ago)	10m
dashboard-5c5b4fff9-zfhdd	1/1	Running	0	66m
locust-55884ffff5-cv6dg	1/1	Running	5 (6m49s ago)	8m40s
mongo-0	1/1	Running	0	27m
postgres-0	1/1	Running	0	81m
redis-6f5fd98786-qh7tg	1/1	Running	0	81m

A) Pods running inside the cluster

```
prakash@SRIDHARANS-LAPTOP18:~/IMS-Prakash$ kubectl get svc -n ims
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
backend	NodePort	10.96.150.18	<none>	8000:30080/TCP	82m
dashboard	NodePort	10.96.105.218	<none>	8501:30081/TCP	82m
locust	NodePort	10.96.190.249	<none>	8089:30089/TCP	82m
mongo	ClusterIP	None	<none>	27017/TCP	82m
redis	ClusterIP	10.96.95.40	<none>	6379/TCP	82m

B) Services Exposed inside the cluster

5) Dashboard Output (Frontend)



← → ↻ 🏠 ⓘ http://localhost:8600

Controls

☒ Auto-refresh

Backend: <http://backend:8000>

Refresh now

Live Feed Closed Incidents **Incident Detail** RCA Form

Work item ID

4298

```
{
  "work_item": {
    "id": 4298
    "component_id": "CACHE_CLUSTER_01"
    "component_type": "CACHE"
    "severity": "CRITICAL"
    "state": "OPEN"
    "start_time": "2026-05-05T10:19:26.048334Z"
    "end_time": NULL
    "mttr_seconds": NULL
    "signal_count": 5
    "summary": "CACHE signal: CRITICAL"
    "created_at": "2026-05-05T10:22:20.804665Z"
    "updated_at": "2026-05-05T10:22:28.879856Z"
  }
  "rca": NULL
}
```

B) Incident Details

Live Feed Closed Incidents **Incident Detail** RCA Form

Work item ID (RCA)

4298

Category

CODE_DEFECT

Fix applied

Restarted service

Prevention

Added retry logic

Submit RCA

C) RCA Form